FinanceFlow

Track Today. Build Tomorrow.

PRODUCT REQUIREMENTS DOCUMENT

FinanceFlow

An implementable, production-minded personal finance web application.

Version 1.2 | 15 October 2026 | Branded dark-mode edition; product scope unchanged

Scope: individual accounts, authentic financial records, insights and recurring financial planning. All monetary figures are illustrative. No bank connectivity in the initial release.

Document map

Area	Coverage
1–3	Vision, branding, user roles and product principles
4–7	Navigation, authentication, dashboard and transactions
8–12	Categories, budgets, recurring items, goals and analytics
13–16	Settings, UX, data rules, backend and security
17–20	API contract, testing, milestones, quality and acceptance

1. Executive product brief

FinanceFlow helps an individual understand where money comes from, where it goes, what remains available and whether spending aligns with planned limits. The product is a responsive, authenticated web application with persistent user-specific records. It must work as a genuine deployed service rather than a static portfolio demo.

1.1 Problem and audience

Primary audience: people managing day-to-day finances, salary, irregular income, bills and short- or long-term saving goals. Users want clarity without accounting jargon. V1 is optimized for individuals, not shared household budgets, bank aggregation, business bookkeeping or investment advice.

1.2 Success measures

Measure	Initial target / definition
Activation	New user creates account and records first transaction
First value	Dashboard reflects first income and expense correctly
Retention proxy	User returns to record activity in a subsequent week
Correctness	Reconciled totals match underlying transaction records
Trust	No cross-user data access, secure recovery and clear deletion flows
Usability	Record a basic transaction in under 45 seconds (target, to validate)

1.3 Scope boundaries

- Must have in first public release: signup/login, password reset, verified email, protected routes, transactions CRUD, categories, monthly budgets, dashboard, core reports, account settings and data export.
- Next release: recurring rules, savings goals, richer analytics, mobile polish and keyboard shortcuts. Architecture should accommodate these from the outset.

Scope boundaries (continued)

Later: multi-currency conversions, household sharing, bank syncing, receipts, notifications and native applications. These are not launch requirements.

2. Branding and identity

2.1 Brand system - dark mode only

FinanceFlow is a calm, practical money-management workspace. Tagline: "Track Today. Build Tomorrow." Clear, secure, unpretentious and proactive; plain language, precise errors and non-judgmental spending feedback.

Dark mode is required on public and authenticated pages at every screen size.
No light theme or theme selector. These tokens replace the earlier light palette.

Token	Hex	Usage
Page background	#0C1723	Main canvas; replaces off-white #F7F9FB
Elevated surface	#142433	Navigation, sidebar and raised sections
Card surface	#192D3C	Cards, fields and nested panels
Subtle border	#2A4050	Dividers, card edges and chart gridlines
Primary text / logo	#EDF5F6	Soft white; headings and the word "Finance"
Secondary text	#A9BDC9	Supporting text; replaces slate #516773
Evergreen action	#077D68	Primary button fill; keep soft-white labels
Mint accent	#5EEAD4	Links, focus rings, positive text and icons
Dark teal highlight	#10352F	Callouts; replaces pale mint #DFF4EC
Sky blue	#377FB5	Retained for chart fills; use text labels too
Bright blue	#60A5FA	Informational text and icons on dark surfaces
Critical / expense	#F87171	Errors, destructive labels and negative warnings
Critical surface	#3B2028	Subdued error backgrounds; replaces light red
Warning	#FBBF24	Near-limit warnings with explicit text labels
Warning surface	#382D16	Subdued near-limit warning background

Old navy #13263B is no longer a text color; #B64242 is replaced for dark-mode warnings.

Inter or system UI; 16px body, 24-32px section headings; tabular financial figures.
8px spacing grid; 8-12px controls; 16px cards; consistent 1.5-2px stroke icons.
Use visible focus and non-color status labels; validate WCAG 2.2 AA contrast in the UI.

2.2 Logo and its usage

Use the supplied flowing F / forward-arrow mark, not a rounded-square redesign.
Change only "Finance" in the wordmark to #EDF5F6; retain the green "Flow" and F motif.
For the full logo, use #A9BDC9 for the tagline if its current color lacks contrast.
Use horizontal logos for landing/sign-in/headers; icon-only for favicon/compact sidebar.
Keep original proportions, transparent background and 25% mark-height clear space.
Provide SVG exports when implementing. This PDF uses the updated supplied logo assets.

2.3 Interface behavior

Desktop app: 240px sidebar, top actions, cards and tables. Tablet: collapsible navigation.
Mobile: drawer navigation, stacked sections/cards and readable ledger rows.
Public landing: preserve the approved mockup content and order across breakpoints.
Empty states guide the next action; loading uses skeletons or explicit progress labels.

3. Roles, accounts and privacy model

V1 has one role, “account owner.” Every account owns its transactions, categories, budgets, recurring rules, goals and settings. There is no admin access to individual finance data within the app UI. Application operators may require audited infrastructure access; do not build a universal admin data browser.

Actor	Allowed actions
Visitor	View marketing/landing pages; start signup; request reset
Authenticated user	Manage only their own data; export or delete account
Unauthenticated client	Access no finance API endpoint; receives 401
Other authenticated user	Must never read/edit another owner’s records; receives non-revealing 404/403

4. Information architecture

Public routes: /, /sign-up, /login, /forgot-password, /reset-password, /verify-email. Protected routes: /app/dashboard, /app/transactions, /app/categories, /app/budgets, /app/recurring, /app/goals, /app/reports and /app/settings. Provide /404 and a generic error route.

Primary navigation	Core contents
Dashboard	Balance summary, monthly cards, category breakdown, recent activity
Transactions	Ledger, search/filter/sort, create/edit/delete, details
Categories	System presets plus user-defined labels
Budgets	Monthly caps, progress, warning states
Recurring	Scheduled future income/expenses and history
Goals	Savings targets and progress contributions
Reports	Period analysis, trends, category breakdown, export
Settings	Profile, security, formatting, data management

Route guards must be backed by server authorization. A hidden link is never a security boundary.

5. Authentication and onboarding

5.1 Registration

Purpose. Create a unique individual account and establish a trusted identity.

- Fields: name, email, password and confirmation; consent to Terms and Privacy Policy where legally appropriate. Normalize email for comparison, not by silently rewriting display preference.
- Validate email format; enforce password policy (suggest 12+ characters, allow long passphrases) and breach-password screening if feasible. Do not require arbitrary special-character composition.
- Create user server-side, hash password with Argon2id (or configured bcrypt alternative) and initiate verification email. Never store or log plaintext passwords.

- After signup, show clear “verify email” state with resend control and rate limiting. Decide and enforce whether restricted app access is allowed before verification.
- On first authenticated visit, onboarding asks base currency (MXN default option), preferred month view and whether the user wants sample walkthrough content; never silently insert fake financial records.

Acceptance criteria:

- PASS — Duplicate addresses do not expose whether an account exists beyond carefully chosen UX
- PASS — New account can verify email using single-use, expiring token
- PASS — Form errors are available to keyboard and screen-reader users

Edge cases and validation:

- Expired verification token; multiple signup attempts; leading/trailing spaces; email service failure

5.2 Login, sessions and logout

Purpose. Protect account access while keeping the login flow understandable.

- Credentials: email/password, with show/hide password control and proper autocomplete attributes.
- Prefer server-managed sessions in Secure, HttpOnly, SameSite cookies; never store long-lived access tokens in localStorage. Use CSRF defense appropriate to cookie auth.
- Login returns minimal current-user information; session endpoint supports refresh/reload. Enforce session expiry and server-side invalidation on logout.
- Apply rate limiting and credential-stuffing defenses without leaking whether an email exists. Log security events without sensitive inputs.
- After successful login redirect to intended protected route where safe; otherwise dashboard.

Acceptance criteria:

- PASS — Invalid credentials produce generic error
- PASS — Refreshing protected page keeps a valid user signed in
- PASS — Logout invalidates server session and private cached data

Edge cases and validation:

- Expired session during mutation; duplicate tabs; authentication request outage

5.3 Password reset, verification and account security

Purpose. Allow secure recovery without leaking account existence.

- Forgot-password accepts email and always returns a neutral confirmation message.
- Reset token is random, stored hashed where practicable, short-lived and single-use; successful reset revokes applicable sessions.
- Allow change password from settings after reauthentication. Display last changed date if recorded.
- Provide security event feedback for password reset and account deletion. Optional MFA is post-V1.

Acceptance criteria:

- PASS — Used/expired token cannot reset twice
- PASS — Unauthorized reset attempts fail
- PASS — Password reset never emails the existing password

Edge cases and validation:

- Reset request spam; user opens two reset links; same password attempt

5.4 Account deletion and data export

Purpose. Give users control of their sensitive records.

- Settings offers downloadable CSV export with transactions plus optionally zipped structured JSON for all personal entities.
- Account deletion requires deliberate confirmation and recent authentication; clearly state permanence and effective data-retention policy.
- Delete or anonymize child finance data according to documented retention rules, invalidate sessions, revoke active tokens and avoid orphaned records.
- Never include password hashes or session tokens in exported files.

Acceptance criteria:

- PASS — Export scoped to owner
- PASS — Deletion removes access immediately
- PASS — UI accurately explains deletion versus backup retention

6. Dashboard

6.1 Monthly overview

Purpose. Answer “How am I doing this month?” within seconds.

- Top bar contains month selector, currency, create-transaction CTA and optional greeting.
- KPI cards: monthly income, monthly expenses, net cashflow (income minus expenses), budget utilization and savings contributions when enabled.
- Use a spending-by-category donut/bar chart, income-versus-expense trend and five most recent transactions.
- Make every card drill down to a filtered transaction or budget view. Empty accounts show a guided first-transaction state.
- Amounts derive from persisted transactions; do not separately maintain dashboard totals in client state.

Acceptance criteria:

- PASS — Switching selected period updates all widgets consistently
- PASS — Refunds, transfers and excluded categories follow explicit calculation rules
- PASS — Loading and no-data states are distinct

Edge cases and validation:

- Zero income; only expenses; no budget exists; high number of categories

6.2 Financial definitions

Term	Rule
Income	Transactions with type=INCOME; positive minor-unit amount
Expense	Transactions with type=EXPENSE; positive minor-unit amount, displayed as outflow
Net cashflow	Sum(income) minus sum(expense) over selected month
Balance (V1)	Cumulative net recorded transactions; label clearly as “Recorded balance”, not bank balance

Budget spent	Eligible expense transactions in the budget's calendar month and category
Savings goal progress	Explicit contributions, not automatically inferred from positive cashflow
Transfer (future)	Excluded from income/expense totals to prevent double counting

7. Transactions: core ledger

7.1 Transaction creation and editing

Purpose. Fast, reliable financial record entry.

- Required: type (income/expense), amount, category, effective date. Optional: description, merchant/payee, notes and tags (tags post-V1).
- Amount input honors locale formatting but persists integer minor units and ISO currency. Disallow NaN, zero when policy disallows it, absurdly large numbers, and >2-decimal MXN entries.
- Date defaults to today in user timezone, not UTC by accident. Allow backdated entries and future dates only with explicit policy.
- Category selector is filtered by transaction type. User-defined category is created inline optionally in V1.1.
- Save button disables while submitting, returns success feedback and refreshes transaction summary. Clear error retains user-entered values.

Acceptance criteria:

- PASS — Created transaction appears once in ledger and totals
- PASS — Edit updates list and dashboard accurately
- PASS — Duplicate submissions are prevented or safely idempotent

Edge cases and validation:

- Slow network; timestamp boundary; currency mismatch; deleted category

7.2 Transaction list and detail

Purpose. Provide an auditable searchable record of what was entered.

- Desktop table columns: date, description, category, type, amount and actions. Mobile uses stacked ledger items.
- Search description/merchant; filters by date range, transaction type, category and amount band. Combine filters using AND; show active filter chips and clear-all.
- Offer sortable date and amount; default newest effective date first, then most recently created. Support server pagination of 20–50 records.
- Edit via detail panel/modal or dedicated route; delete requires confirmation, supports clear success/error messaging.
- Provide export for currently selected period and an all-time export; state explicitly which one is generated.

Acceptance criteria:

- PASS — Filter combinations are stable and URL-shareable where appropriate
- PASS — Pagination remains correct after deletion
- PASS — Authorization checked for every item ID

Edge cases and validation:

- Empty filters; deleted item open in another tab; duplicate description

7.3 Transaction business rules

Purpose. Keep calculations predictable.

- Transactions are always owned by a user and reference an allowed category owned by them or an immutable system category.
- A user cannot attach another user's category to a transaction, even if they guess an ID.
- Use explicit decimal-to-minor-unit conversion on backend; never use JS floating-point arithmetic for stored money.
- Prefer soft-delete only if retention/audit requirements justify it; otherwise hard-delete with clear cascading rules.
- Keep createdAt and updatedAt independent from effective transaction date.

Acceptance criteria:

- PASS — Boundary inputs produce deterministic rounding/rejection
- PASS — Aggregates agree with ledger results
- PASS — Invalid foreign keys are rejected

8. Categories

8.1 Category management

Purpose. Let users group spending without corrupting historical data.

- Provide starter expense categories such as Housing, Groceries, Transport, Bills, Dining, Health, Entertainment, Shopping and Other; starter income categories Salary, Freelance, Gifts, Other.
- Each category has ID, owner or system designation, name, type, icon and optional color. User can create, rename, recolor and archive their own categories.
- Do not allow editing or deleting global system categories directly. Archive rather than delete categories referenced by transactions.
- Archived categories remain visible on old records but cannot be selected for new records unless reactivated.

Acceptance criteria:

- PASS — Names are unique per owner/type under a defined normalization policy
- PASS — Archiving never erases history
- PASS — Category totals update when a transaction is recategorized

9. Monthly budgets

9.1 Budget planning

Purpose. Make spending limits actionable without blocking spending.

- User sets a monthly spending limit per expense category and optionally an overall monthly limit.
- Month/year selector controls budget period. Defaults can be copied from previous month, but copying does not overwrite existing budgets without confirmation.

- Budget card displays limit, spent, remaining, percentage, visual progress and over-budget amount.
- Warning thresholds: e.g. >=80% amber and >100% red, with text labels; crossing a limit must not prohibit recording expenses.
- When a transaction is edited or deleted, calculated budget spending updates automatically.

Acceptance criteria:

- PASS — One category budget per user/category/month
- PASS — Past budget snapshots remain correct under chosen ledger policy
- PASS — Zero limit has explicit UX

Edge cases and validation:

- Overspend; backdated expense; archived category; leap-year February

10. Recurring transactions (Release 1.1)

10.1 Recurring schedules

Purpose. Represent rent, salary and subscriptions without creating duplicates.

- Rule fields: type, amount, currency, category, description, start date, optional end date, frequency (weekly, monthly, yearly), next due date, status and reminder preference.
- Monthly recurrence for a 29th/30th/31st must state whether it uses the last day of short months. Choose last valid calendar day.
- For V1.1, generate “upcoming expected items” and require manual confirmation to post a real ledger transaction. Optional automatic posting can be a later opt-in.
- Users can pause, resume, edit and terminate a rule. Editing future instances must not silently rewrite already confirmed ledger records.
- Background processing must be idempotent and timezone-aware; use stable occurrence IDs/unique constraint to prevent duplicates.

Acceptance criteria:

- PASS — Monthly boundary dates behave consistently
- PASS — Rule pause stops new occurrences
- PASS — Confirming an occurrence creates one ledger record

Edge cases and validation:

- Catch-up after downtime; missed confirmation; user changes timezone

11. Savings goals (Release 1.1)

11.1 Goal tracking

Purpose. Make saving objectives visible without confusing them with bank balances.

- Goal fields: name, target amount, optional target date, description, status and manually entered contributions.
- Goal progress is sum of contribution records, not a subtraction from ledger balance by default. Explain that contributions are tracking entries, not money transfers.
- Offer add/remove/edit contribution, progress percentage, estimated remaining amount and completed state.
- An optional transaction link can be a later feature, but it must avoid double counting.

Acceptance criteria:

- PASS — Goal progress is never silently inferred from expenses
- PASS — Editing a contribution recalculates exactly
- PASS — Goals can exceed target without corrupting sums

12. Reporting and insights

12.1 Reports

Purpose. Help users identify trends, not merely display charts.

- Provide date controls: this month, previous month, last 3 months, year-to-date and custom inclusive range.
- Charts: monthly income/expense bars, category expense distribution, top spending categories and period-over-period differences.
- Each chart has accessible text or tabular equivalent and clearly shows currency, date range and definitions.
- CSV uses explicit headers, ISO dates, currency and amounts with documented decimal representation. Escape formula-like input when opening spreadsheets.
- No prescriptive “financial advice” suggestions in V1; use factual observations such as “Groceries were 18% of recorded expenses.”

Acceptance criteria:

- PASS — Reports reconcile with filtered ledger
- PASS — No data produces explanation rather than empty broken chart
- PASS — Exports preserve owner restriction

Edge cases and validation:

- Currency changes; hidden/archived categories; very long date ranges

13. Account settings

13.1 Profile, localization and preferences

Purpose. Manage user identity and presentation.

- Edit display name and preferred currency; changing preferred display currency does not magically convert historical transactions.
- Offer locale/date format and timezone choice. V1 may constrain all transactions to one base currency per account, with visible reason.
- Security tab: change password, logout all sessions if supported, reset link, verification status.
- Data tab: CSV export, download account data and destructive account deletion flow.
- Appearance: dark mode only on desktop, tablet and mobile; no light theme or theme toggle.

Acceptance criteria:

- PASS — Invalid settings inputs are rejected with specific messages
- PASS — Preferences survive reload
- PASS — Sensitive changes are authenticated

14. UX states, accessibility and responsiveness

State	Required handling
Loading	Skeleton/table loader, disable duplicate submission, no misleading zero totals
Empty new account	Friendly onboarding, primary “Add your first transaction” button
Empty filtered results	“No matching transactions”; clear filters
Validation error	Inline associated error; summary for multiple fields
API failure	Readable retry; preserve unsaved values
Destructive action	Confirm intent, clearly identify affected item
Success	Non-blocking status/confirmation and refreshed data
Session expired	Prompt login and handle unsaved data gracefully

Target WCAG 2.2 AA as practical acceptance objective: keyboard operation, visible focus, sufficient contrast, semantic labels, form errors, chart alternatives and reduced-motion respect. At 320px viewport, no critical controls become inaccessible. Support recent Chrome, Safari, Firefox and Edge.

15. Technical architecture and repository

Suggested monorepo structure: /apps/web (React + TypeScript + Vite), /apps/api (Node + Express + TypeScript), /packages/shared (Zod request/response schemas/types when useful), /infra (deployment documentation), /docs (decisions and screenshots). Avoid introducing monorepo tooling until ordinary npm workspaces are genuinely helpful.

Concern	Recommended baseline
Frontend	React, TypeScript, Vite, React Router
Styling	CSS modules or Tailwind; choose one consistent approach
Local state	useState first; Context/useReducer for instructive shared case
Shared client state	Zustand when currency/preferences warrant it
Server data	Fetch + effects initially; TanStack Query during deliberate refactor
Forms	Controlled inputs first; React Hook Form + Zod after basics
API	Express, TypeScript, Zod validation
Persistence	PostgreSQL with Prisma ORM and migrations
Auth	Server session cookies + secure credential handling
Tests	Vitest + React Testing Library, API integration tests, Playwright smoke E2E
CI	GitHub Actions lint/typecheck/test on pull requests
Hosting	Any reputable service with TLS, managed Postgres and secret management

15.1 State ownership rule

- Local ephemeral state: modal, form draft, table row expansion -> useState.
- Shared UI settings: currency display, sidebar preference -> Context or Zustand as complexity warrants.
- Server state: transactions, categories, budgets, authenticated profile -> TanStack Query cache, not a duplicated Zustand copy.
- URL state: search query, selected month, filters and pagination when shareability/navigation matter -> URL search params.

15.2 Critical backend boundaries

- Every protected route authenticates session and derives owner ID from server session; never trust userId sent by browser.
- Use database constraints, input validation, scoped queries and centralized error handling. Parameterize any raw SQL.
- Store money in integer minor units as BIGINT; return string minor-unit values to JS clients if size could exceed safe integer. Currency must be explicit.
- Write UTC timestamps for instants and date-only values for effective financial dates where appropriate; derive month from user-defined calendar behavior.
- Use migrations and seed only shared non-sensitive defaults. Provide separate local/test/prod configurations.

16. Data model

Entity	Essential fields / relations
User	id, name, normalizedEmail UNIQUE, passwordHash, verifiedAt?, baseCurrency, timezone, locale, createdAt, updatedAt
Session	id, userId FK, sessionTokenHash UNIQUE, expiresAt, revokedAt?, createdAt
VerificationToken	id, userId, tokenHash, purpose, expiresAt, consumedAt?
Category	id, ownerId? (null for global), type, name, icon, color, archivedAt?, createdAt
Transaction	id, userId FK, categoryId FK, type, amountMinor BIGINT, currency, effectiveDate DATE, description, merchant?, notes?, createdAt, updatedAt
Budget	id, userId FK, categoryId FK nullable for overall, periodMonth DATE, limitMinor BIGINT, currency, createdAt, updatedAt
RecurringRule	id, userId FK, categoryId FK, amountMinor, type, cadence, startDate, endDate?, nextDate, status
RecurringOccurrence	id, ruleId FK, scheduledDate, transactionId?, status, unique(ruleId,scheduledDate)
Goal	id, userId FK, title, targetMinor, currency, targetDate?, status
GoalContribution	id, goalId FK, amountMinor, contributedOn, notes?

Indexes: transactions(userId,effectiveDate DESC,id); transactions(userId,categoryId,effectiveDate); budgets(userId,periodMonth); sessions(tokenHash); recurring rules(userId,nextDate). Unique budget(userId,categoryId,periodMonth), with a separate rule for null overall category. Foreign keys and cascades must be reviewed so account deletion and category archiving behave intentionally.

17. REST API contract

Base /api/v1; JSON request/response, standardized {data, meta?} successes and {error: {code,message,fieldErrors?}} failures. Validate all input on server. Version APIs deliberately. Use 400 validation, 401 unauthenticated, 403 forbidden when appropriate, 404 missing/out-of-scope, 409 conflict, 429 rate-limited and 500 unexpected error.

Method + path	Behavior
POST /auth/register	Create account and send verification
POST /auth/login	Create server session
POST /auth/logout	Revoke current session
GET /auth/me	Authenticated minimal profile
POST /auth/forgot-password	Issue non-enumerating reset request
POST /auth/reset-password	Consume token and set new password
POST /auth/verify-email	Consume verification token
GET /transactions	Paginated scoped list with filters/sort
POST /transactions	Validate and create ledger item
GET /transactions/:id	Return owner-scoped details
PATCH /transactions/:id	Update approved fields only
DELETE /transactions/:id	Delete owner-scoped item
GET /categories	List system + own active categories
POST /categories	Create owned category
PATCH /categories/:id	Edit/archive own category
GET /budgets?month=YYYY-MM	Return budget limits and computed spend
PUT /budgets/:id	Upsert/update budget as designed
GET /dashboard?month=YYYY-MM	Aggregate income/expense/category and recent
GET /reports?from=&to=	Return report aggregates
GET /exports/transactions.csv	Return authorized CSV download
GET/PATCH /settings	Read/update allowed preferences
DELETE /account	Confirm and delete account safely
GET/POST/PATCH /recurring	Release 1.1 recurring rules
GET/POST/PATCH /goals	Release 1.1 goal records

17.1 Sample transaction payload

POST /api/v1/transactions ->
{"type":"EXPENSE","amountMinor":"12990","currency":"MXN","categoryId":"cat_uuid","effectiveDate":"2026-10-

15","description":"Groceries","merchant":"Local market"}. Response includes id, owner-scoped fields and server timestamps. Never accept the owner ID from client.

17.2 API robustness

- GET accepts limit bounded by server (e.g. 100 maximum), cursor or page token, and strict allowed sort keys. Avoid unbounded result sets.
- Mutations can use an Idempotency-Key for retry-prone creation where practical; client must avoid double-submit.
- Audit logs note actor/action/entity and outcome without including raw financial notes or passwords.

18. Security, trust and operations

- Enforce TLS everywhere in deployment; use secure cookies, HTTP security headers, trusted proxy configuration and environment secrets outside source control.
- Defense in depth: password hash, CSRF protection, XSS avoidance, CORS allowlist where relevant, rate limits, request size limits, robust input parsing.
- Privacy: least-privilege DB credentials, no customer financial values in analytics/traces, documented retention/deletion and backup policies.
- Protect exports as sensitive downloads; prevent browser/proxy caching of sensitive API responses where appropriate.
- Use structured logs with request IDs, health check endpoint, monitored errors, periodic database backups and tested restoration.
- Establish legal privacy/terms language appropriate to jurisdiction before public launch; do not claim bank-grade certification or financial regulation compliance without evidence.

18.1 Threat cases to explicitly test

- User A guesses User B transaction ID; API must not reveal data.
- Expired/revoked session issues a POST request; must fail.
- Malicious notes contain HTML/script or spreadsheet formula; safely render/escape/export.
- Concurrent budget writes and duplicate POSTs; enforce server/database invariants.
- Password reset requested repeatedly; responses do not enumerate accounts.

19. Testing strategy and definition of done

Level	Example checks
Unit	Formatting money; period boundaries; calculations; filter and sort functions
Component	Transaction form validation; loading/error/empty; keyboard navigation
API integration	Auth lifecycle; ownership; CRUD; validation; month aggregates
E2E	Signup -> verify -> login -> add income/expense -> budget -> export -> logout
Security	Cross-account IDOR, CSRF, rate limits, session revocation

Responsive	320px, tablet, desktop and keyboard/screen-reader smoke checks
Performance	Index-driven queries, pagination and reasonable dashboard response with seeded data

Definition of done for every feature: designed state(s), accessible UI, typed data, server validation, authorization, automated meaningful tests, documented acceptance checks, error/empty/loading states and reviewed code. A screenshot alone is not a completed feature.

20. Build sequence: vertical slices for learning

Do not build a huge mock-only frontend and throw it away. Create thin end-to-end slices. Keep ~70% of deliberate practice on React/frontend and ~30% on backend because interview preparation is the priority.

Slice	Deliverable	React learning objective
0: Foundations	Repo, app shell, Postgres, API health, test baseline	Routing, component organization, TS config
1: Auth	Register/login/me/logout and protected dashboard	Forms, controlled state, async flow, route guards
2: Read ledger	Seed real transactions; GET API; list with loading/error	useEffect, cleanup, lists, typed responses
3: Edit ledger	Create/edit/delete real transactions	Controlled forms, optimistic vs refetch decisions
4: Navigate ledger	Filters/sort/search/pagination using URL	Derived state, URL state, memoization judgement
5: Dashboard	Aggregates and meaningful charts	Shared components, data transformation
6: Budgets	Create/update limits, live progress	Cross-page coordination and component tests
7: State refactor	Introduce Zustand for legitimate shared preferences	State ownership decisions
8: Server cache	Introduce TanStack Query for API records	Caching, invalidation, mutation lifecycle
9: Ship V1	Reports/export, reset, tests, deploy	Production hardening and interview explanation
10: Extensions	Recurring, goals and richer reports	Data integrity, state complexity

20.1 Recommended personal learning constraints

- Implement new UI logic yourself before asking AI for a solution. Documentation and targeted hints are allowed.
- For every hook, explain dependency behavior, render triggers, data ownership and why derived data is or is not state.
- After each slice, fix one intentional bug without AI, add one test, and write a 3-sentence architectural explanation.
- Practice time-boxed tasks: add a filter, repair stale results, implement accessible form errors, update a failing test.

20.2 MVP release gate

- User can create, verify, sign in, reset password and sign out securely.
- User only ever sees their own finances.
- User can add/edit/delete/search transactions with correct monetary totals.
- User can categorize activity and set monthly budgets.
- Dashboard and reports reconcile with the transaction ledger.
- User can export data and delete account.
- Core flows work on phone and desktop with accessible controls.
- Deployed application has TLS, secrets, database backups and smoke tests.

Appendix A. Product decisions to lock before coding

- Do we allow transactions dated in future, or represent future entries only as recurring expectations?
- Should accounts be single-currency in V1? Recommendation: yes, MXN or USD chosen once, with an explicit migration strategy later.
- Is recorded balance cumulative from first recorded transaction, or is there an optional opening balance? Recommendation: support a clearly labeled opening balance in onboarding as a special entry.
- Can global categories be archived by user preference? Recommendation: hide per user without editing globally.
- Do we require verified email before any finance data entry? Recommendation: yes for public deployment.
- Do budgets cover only expense transactions? Recommendation: yes.
- Are recurring entries auto-posted? Recommendation: no, require confirmation in 1.1.
- What are actual privacy disclosures, hosting region and retention rules prior to public launch?

Appendix B. User stories

- As a first-time user, I can register and verify my email so my financial records are private.
- As a returning user, I can see this month’s income, expenses and recorded net within seconds.
- As a user, I can record yesterday’s purchase correctly and have my monthly budget adjust.
- As a user, I can find transactions for a category and date range without scrolling through everything.
- As a user, I can set a grocery limit and understand whether I am approaching or exceeding it.
- As a user, I can see my recurring rent before it is due and confirm when I paid it.
- As a user, I can export my records and permanently delete my account.